Fast Entailment Checking for a fragment of Separation Logic

Quang-Trung Ta, Andreea Costea and Siau-Cheng Khoo
Department of Computer Science, National University of Singapore

NUS National University of Singapore

Introduction

Separation Logic is known for its application in reasoning about programs manipulating heap memory. One of its main challenges is the entailment checking [2,3,4,5]. Following this line of research, we focus on a "linear fragment" of Separation Logic, and work towards improving the performance of the entailment checking involving user-given predicates.

Motivation

Given a program to be verified…

```
struct node { int value; struct node* next; }

void concat (struct node** x, struct node** y) {
  if (*x == NULL) *x = *y;
  else {
    struct node* tmp = *x;
    while (tmp->next != NULL)          // (VC-1)
      tmp = tmp->next;
    tmp->next = *y;                    // (VC-2)
  } }
```

… and its corresponding formal specification using Separation Logic…

Pre- and post-condition of *concat* procedure:

```
requires (*x)↦lseg_even<NULL> ∗ (*y)↦lseg_odd<NULL>
ensures  (*x)↦lseg_odd<NULL>
```

In which:

```
lseg_odd<p>    ≡ (self↦node<_,p>)
(list segment with odd length)   ∨ (self↦node<_,r> ∗ r↦lseg_even<p>)

lseg_even<p>   ≡ (self = p)
(list segment with even length)  ∨ (self↦node<_,r> ∗ r↦lseg_odd<p>)
```

… a program verifier generates the following verification conditions in form of Separation Logic Entailments:

```
(VC-1): (*x)↦lseg_even<NULL> ∧ (*x)≠NULL  ⊨  (*x)↦lseg_odd<u> ∗ u↦node<_,NULL>
(VC-2): (*x)↦lseg_odd<u> ∗ u↦node<_,v> ∗ v↦lseg_odd<NULL>  ⊨  (*x)↦lseg_odd<NULL>
```

How to automatically check these entailments?

Method

Step 1: Discover *normalized predicates* (the most general predicates which can be used for describing all user-given predicates)

```
lseg<p,n> ≡ (self = p) ∨ (self↦node<_,r> ∗ r↦lseg<p,n-1>).
```

Step 2: Transform the user-given predicates to their corresponding normalized forms.

```
lseg_even<p> ⇔  lseg<p,n> ∧ (∃k: k≥0 ∧ n=2*k)
lseg_odd<p>  ⇔  lseg<p,n> ∧ (∃k: k≥0 ∧ n=2*k+1)
```

Step 3: Normalize the entailments using the predicates obtained in step 2:

```
(VC-2): (*x)↦lseg_odd<u> ∗ u↦node<_,v> ∗ v↦lseg_odd<NULL>
                    ⊨   x↦lseg_odd<NULL>

  ⇒   (∃k,h: (*x)↦lseg<u,n> ∗ u↦node<_,v> ∗ v↦lseg<null,m>
                        ∧ k≥0 ∧ n=2*k+1 ∧ h≥0 ∧ m=2*h+1)
                ⊨  (∃t: (*x)↦lseg<null,s> ∧ s=2*t+1)
```

Step 4: Check validity of the normalized entailments. Our decision procedure employs the following techniques:

- *Match data nodes:*
  `x↦node<_,u> ∗ … ⊨  x↦node<_,v> ∗ …`
- *Subtract predicates:*
  `x↦lseg<u,n> ∗ … ⊨  x↦lseg<v,m> ∗ …`
- *Unfold predicates:*
  `x↦lseg<u,n> ∗ … ⊨  x↦node<_,v> ∗ …`
- *Accelerated unfold predicates:*
  `x↦lseg<u,n> ∗ …`
  `⊨  x↦node<_,v1> ∗ v1↦node<_,v2> ∗ v2↦node<_,v3> ∗ …`
- *Approximate to pure formulas:*
  `x↦lseg<u,n> ∗ u↦node<_,_> ∗ …`
  `⊨ x≠u ∧ u≠NULL ∧ …`

Discussion

- **Separation Logic fragment of interest:** linear fragment (involving singly linked list, doubly linked list, cyclic linked list, linked list with even or odd length…)
- **Sound and complete**
- **Efficient**: courtesy to our novel accelerated unfolding mechanism and the usage of normalized predicates
- **Expressive**: our decision procedure is able to reason about linear predicates with user-given pure properties
- Please note that this is an ongoing work, implemented on top of our existing program verification framework, namely HIP/SLEEK [1].

References

[1] Chin et. al., "HIP/SLEEK project": http://loris-7.ddns.comp.nus.edu.sg/~project/hip/index.html
[2] Enea et. al., "Compositional Entailment Checking for a Fragment of Separation Logic", APLAS'14
[3] Iosif et. al., "The tree width of separation logic with recursive definitions", CADE'13
[4] Piskac et. al., "Automating separation logic using SMT", CAV'13
[5] Pérez et. al., "Separation Logic Modulo Theories", APLAS'13

Contacts: {taqt, andreeac, khoosc}@comp.nus.edu.sg